

MoSAIC: Modular Self-Attentive Interacting Columns for Recurrent Memory

Anonymous Submission

Abstract

Recurrent sequence models compress history into a fixed-size state. Increasing their width adds capacity, but leaves the organization and exchange of information within that state implicit. We introduce MoSAIC (Modular Self-Attentive Interacting Columns), a recurrent architecture that distributes a parameter budget across a grid of independently parameterized GRU columns. At every timestep, the columns use learned attention to read from a small message bank. The first layer reads both the current input and the previous top-layer column states, while each subsequent layer reads the newly updated states below it. Attention therefore determines what each column receives before its recurrent update, and the complete top layer is carried forward as a compact recurrent communication workspace.

We study whether this explicit modular routing improves sequence modeling under matched parameter budgets. Our evaluation compares MoSAIC with depth-matched and parameter-matched GRUs and modern recurrent and attention-based baselines on character-level language modeling and controlled memory tasks. We additionally measure the effects of grid width, delayed feedback, column identities, and routing regularization, while reporting recurrent-state size and computational cost alongside predictive quality.

Introduction

Many sequence problems require a model to retain information while continually incorporating new observations. Character-level language modeling tests this ability on natural streams, while controlled associative-recall tasks isolate storage, interference, and retrieval over known delays. Recurrent neural networks are a natural fit for these settings because their inference cost and state size do not grow with the processed sequence length.

Standard GRUs (Cho et al., 2014) and LSTMs (Hochreiter & Schmidhuber, 1997) represent the past with one hidden vector per layer. Widening or stacking these vectors increases capacity, but does not explicitly organize how distinct stateful computations exchange information. Modern linear recurrences instead improve parallel training and long-range propagation (Dao & Gu, 2024; Qin et al., 2024; Yang et al., 2024), yet usually retain a single recurrent stream per layer. This motivates a complementary question: **how should a fixed recurrent parameter budget be organized when several persistent stateful modules are allowed to communicate?**

Modular recurrent architectures provide an important precedent. Recurrent Independent Mechanisms (RIMs) (Goyal et al., 2021) maintain separate recurrent modules, select a sparse subset for each input, and allow the active modules to communicate through attention. BRIMs (Mittal et al., 2020) introduce hierarchical top-down and bottom-up communication, while global-workspace models (Goyal et al., 2022) impose a shared communication bottleneck.

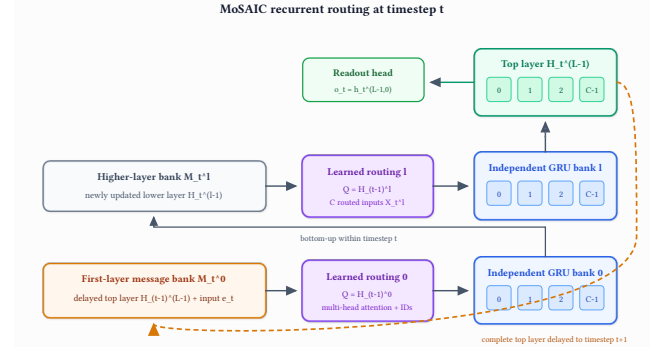


Figure 1: **MoSAIC at timestep t .** The first routing layer is queried by the previous states H_{t-1}^0 and reads a bank containing the delayed top-layer states H_{t-1}^{L-1} and current embedding e_t . Its C messages are processed by independent GRU cells to produce H_t^0 . Each later layer repeats the operation using the newly updated layer below as its message bank. The readout uses column 0 of the top layer, while all top-layer columns are carried to timestep $t+1$.

These methods establish that modular recurrence can be useful, but couple modularity to sparse activation or a particular workspace mechanism. We instead study a simpler dense setting in which every module updates at every timestep and learned routing determines its recurrent input.

We propose **MoSAIC (Modular Self-Attentive Interacting Columns)**, illustrated in Figure 1. MoSAIC replaces

each recurrent layer with C independently parameterized GRU cells. Before a cell updates, its previous state queries a shared message bank through multi-head attention. At the bottom layer this bank contains the current input together with the **delayed complete top layer** from the preceding timestep; at higher layers it contains the current states of the layer below. Consequently, the model retains column-local recurrent states while learning both bottom-up and temporally delayed communication routes.

The architecture deliberately separates its defining computation from optional training regularizers. Learnable query/key identities make communication participants distinguishable. In the configuration used in our main experiments, training-time logit noise, a diagonal-route cost, and an attention-entropy bonus regularize routing, but they do not change the inference-time recurrent graph. We ablate these choices separately.

Our main contributions are:

1. **A recurrent modular architecture** in which independently parameterized GRU columns learn their inputs through attention over bottom-up and delayed top-down message banks.
2. **A controlled fixed-budget evaluation** against monolithic GRUs and modern recurrent and attention-based sequence models, reporting predictive quality together with parameter count, recurrent-state size, and computational cost.
3. **An analysis of modular routing** through width/depth comparisons and ablations of delayed feedback, participant identities, and routing regularization.

Related Work

Multi-Dimensional and Grid RNNs.. Multi-Dimensional RNNs (Graves et al., 2007) recurrently propagate information along multiple axes of structured data. Grid LSTM (Kalchbrenner et al., 2016) extends the idea by arranging LSTM blocks in a multidimensional grid whose dimensions exchange hidden and memory vectors through fixed connections. MoSAIC uses a grid in a different sense: all columns share one temporal axis, retain separate recurrent states, and exchange information through content-dependent routing rather than through edges fixed by the geometry of an input.

Modular Networks and Attention-Based Communication.. RIMs (Goyal et al., 2021) are the closest conceptual predecessor: they preserve independent recurrent mechanisms, sparsely select mechanisms for each input, and allow active mechanisms to communicate through attention. BRIMs (Mittal et al., 2020) organize such mechanisms hierarchically, and shared global workspaces (Goyal et al., 2022) restrict communication through a bandwidth-limited latent. Relational recurrent networks (Santoro et al., 2018) instead update a set of interacting memory slots through self-attention. MoSAIC shares the separation of persistent states and learned communication, but uses

dense synchronous updates: every column queries a layer-specific message bank and then performs its own recurrent update. Its delayed top-layer bank also makes the communication graph recurrent across both layers and time.

Modern Recurrent Sequence Models.. Recent work has substantially strengthened recurrent alternatives to Transformers. LRUs (Orvieto et al., 2023) stabilize long linear recurrences; HGRN2 (Qin et al., 2024) expands the state of a gated linear RNN; Mamba-2 (Dao & Gu, 2024) connects selective state-space models and structured attention; and DeltaNet (Yang et al., 2024) uses a delta-rule matrix state to improve associative recall. xLSTM (Beck et al., 2024) extends LSTM gating and introduces scalar- and matrix-memory variants, including the mLSTM baseline used in our evaluation. These models primarily redesign the recurrent update or its state representation. MoSAIC instead keeps a conventional nonlinear GRU update and changes how a fixed parameter budget is distributed and routed.

Recall Evaluation.. Associative-recall studies show that aggregate language-modeling quality can obscure substantial differences in information retrieval. Zoology (Arora et al., 2023) uses multi-query associative recall to connect synthetic retrieval behavior with language-modeling performance. POPGym (Morad et al., 2023) provides controlled partially observable tasks spanning object, sequential, capacity, and spatial memory. We use character-level language modeling as the principal natural-sequence evaluation and store-distract-query tasks to measure behavior as the retention interval and interference increase.

Preliminaries and Problem Setting

Let $e_t \in \mathbb{R}^{B \times H}$ be an input embedding at timestep t , where B is batch size and H is hidden width. A stacked GRU with L layers maintains $S_t \in \mathbb{R}^{L \times B \times H}$ and computes one recurrent stream per layer. We compare models under a matched trainable-parameter budget, using the same token embedding, readout, data order, optimizer, and number of training tokens.

Parameter matching does not equate every resource. A stacked GRU stores LBH recurrent scalars, whereas an $L \times C$ MoSAIC grid stores $LCBH$. We therefore report the number of recurrent scalars per example, measured throughput, and peak memory in addition to parameters. This separates predictive gains under a fixed weight budget from the activation-memory and compute costs of exposing multiple column states.

Method

Recurrent Grid

MoSAIC maintains $H_t \in \mathbb{R}^{L \times C \times B \times H}$, where $h_t^{l,c}$ is the state of column $c \in \{0, \dots, C-1\}$ at layer $l \in \{0, \dots, L-1\}$. Each layer-column pair has independent GRU parameters. The cells of a layer are evaluated in one batched operation, but do not share recurrent weights.

The first layer reads from a message bank containing the delayed top layer and the current external inputs:

$$\mathbf{M}_t^0 = [\mathbf{H}_{\{t-1\}}^{\{L-1\}}; \mathbf{e}_t^1; \dots; \mathbf{e}_t^I] \in \mathbb{R}^{\{(C+I) \times B \times H\}}.$$

Here I is the number of external input streams; $I = 1$ for language modeling. For every subsequent layer, the message bank is the newly updated layer below:

$$\mathbf{M}_t^l = \mathbf{H}_t^{\{l-1\}}, \quad l > 0.$$

Thus information moves bottom-up within a timestep, whereas the complete top layer supplies delayed feedback across timesteps. Unlike a conventional stacked GRU, a column does not receive the same-index lower-layer state by a fixed edge: it learns a distribution over the available messages.

Learned Routing

At layer l , the C previous column states act as queries. Learnable participant identities $\mathbf{a}_c^Q$ and $\mathbf{a}_j^K$ distinguish query columns and message sources. For attention head r ,

$$\mathbf{q}_{\{t,c,r\}}^l = \text{SiLU}\left(\mathbf{W}_r^Q\left(\mathbf{h}_{\{t-1\}}^{\{l,c\}} + \mathbf{a}_c^Q\right) + \mathbf{b}_r^Q\right),$$

$$\mathbf{k}_{\{t,j,r\}}^l = \text{SiLU}\left(\mathbf{W}_r^K\left(\mathbf{m}_{\{t,j\}}^{\{l\}} + \mathbf{a}_j^K\right) + \mathbf{b}_r^K\right),$$

$$\mathbf{v}_{\{t,j,r\}}^l = \text{SiLU}\left(\mathbf{W}_r^V\mathbf{m}_{\{t,j\}}^{\{l\}} + \mathbf{b}_r^V\right).$$

Each query column has a learned positive inverse temperature $\beta_c = \text{softplus}(\rho_c)$. The routing probabilities are

$$\pi_{\{t,c,j,r\}}^l = \text{softmax}_j\left(\beta_c\left(\frac{\mathbf{q}_{\{t,c,r\}}^l \cdot \mathbf{k}_{\{t,j,r\}}^l}{\sqrt{\frac{H}{R}}} + \varepsilon_{\{t,c,j,r\}}^l\right)\right),$$

where R is the number of heads. The perturbation ε is zero at inference; during training we optionally sample independent zero-mean Gaussian noise. Concatenated head outputs are projected and normalized:

$$\mathbf{x}_{\{t,c\}}^l = \text{LN}\left(\mathbf{W}^O\left[\sum_j \pi_{\{t,c,j,1\}}^l \mathbf{v}_{\{t,j,1\}}^l; \dots; \sum_j \pi_{\{t,c,j,R\}}^l \mathbf{v}_{\{t,j,R\}}^l\right]\right).$$

This attention operation produces the input to the recurrent cell; it does not replace or post-process the cell state.

Independent Recurrent Updates and Readout

After routing, every column performs an ordinary GRU update with its own parameters:

$$\mathbf{h}_t^{\{l,c\}} = \text{GRU}_{\{l,c\}}\left(\mathbf{x}_{\{t,c\}}^l, \mathbf{h}_{\{t-1\}}^{\{l,c\}}\right).$$

The prediction head reads column 0 of the final layer, $\mathbf{o}_t = \mathbf{h}_t^{\{L-1,0\}}$. The recurrent state passed to the next timestep contains every column, and $\mathbf{H}_t^{\{L-1\}}$ becomes part of the next first-layer message bank. Restricting the readout to one column prevents a pooled output from bypassing the learned routes, while leaving all columns available as recurrent memory.

Routing Regularization

The recurrent grid, message banks, and learned routing distribution define MoSAIC. Our main training configuration additionally uses three routing regularizers: Gaussian logit noise, a diagonal-route cost that biases columns toward retaining a low-cost self route, and an entropy bonus that discourages prematurely deterministic routing. At the first layer the cost also encourages the designated readout column to receive the external input while discouraging direct input access for later columns. For task loss L_{task} ,

$$L = L_{\text{task}} + \lambda_{\text{comm}} L_{\text{comm}} - \lambda_{\text{ent}} H(\pi).$$

These terms are applied only during training and add no recurrent state or inference-time operation. We report them as ablations rather than treating them as architectural contributions.

Complexity and Resource Accounting

Ignoring embeddings and biases, independent GRUs contribute approximately $6LCH^2$ parameters and the routing projections contribute approximately $4LH^2$. The recurrent state contains LCH scalars per example. Routing also requires $O(LC(C+I)H)$ score computation per timestep in addition to the dense projections and GRU updates. Because a parameter-matched MoSAIC can therefore retain more state coordinates than a monolithic GRU, our evaluation reports parameters, recurrent-state size, throughput, and peak memory together.

Experiments

Our experiments ask four questions: (Q1) does distributing a fixed parameter budget across routed recurrent columns improve language modeling over a monolithic GRU; (Q2) how should that budget be divided between depth, column count, and within-column width; (Q3) which routing and feedback components are necessary; and (Q4) what activation-memory and computational costs accompany any quality gain?

Character-Level Language Modeling

Data and evaluation. We use text8, a 100-million-character normalized English corpus with a 27-character alphabet. Following the conventional protocol, the first 90M characters are used for training, the next 5M for validation, and the final 5M for test. Checkpoints and hyperparameters are selected using validation bits per character (BPC); test BPC is evaluated once for the selected checkpoint. All models process the corpus in the same contiguous order and use the same tokenization and reset schedule.

Training. Models are optimized with RMSprop and truncated backpropagation through time using 512 parallel streams and a truncation length of 64. The per-token random state-reset probability decays during training, increasing the expected uninterrupted context from approximately 320 to 64,000 characters. Gradients are clipped to norm 1.0. The learning rate warms up from near zero, reaches 5×10^{-4} , and decays to 5×10^{-5} . We

Model	Shape	Params	State	Test BPC ↓
GRU	L1	10.16M	1,296	—
GRU	L2	10.04M	1,824	—
GRU	L3	10.23M	2,256	—
MoSAIC	L1C8	9.39M	3,392	—
MoSAIC	L2C4	10.11M	3,392	—
MoSAIC	L2C8	10.70M	5,120	—
MoSAIC	L2C16	10.09M	7,168	—
MoSAIC	L3C4	8.65M	3,840	—

Table 1: Parameter-matched text8 evaluation. State counts exclude optimizer state and report persistent recurrent scalars per example. Final cells will contain mean $\pm$ standard deviation over seeds.

train each configuration for the same number of observed characters and report mean and standard deviation over independent seeds.

Models and resource matching.. Table 1 lists the currently tested configurations. GRU depth and MoSAIC grid shape are varied while keeping trainable parameters close to 10M. Because parameter matching yields different recurrent-state sizes, the table also reports the number of state scalars per example. We additionally compare against parameter-matched mLSTM (Beck et al., 2024), HGRN2 (Qin et al., 2024), DeltaNet (Yang et al., 2024), and a causal Transformer (Vaswani et al., 2017) using the same data and optimization protocol.

Architectural Ablations

We isolate grid shape using configurations that vary columns at fixed depth and depth at comparable column count. Component ablations remove: (i) participant identities, (ii) delayed top-layer feedback from the first-layer bank, (iii) nonlinear query/key/value projections, (iv) training-time routing noise, (v) the diagonal communication cost, and (vi) the routing-entropy bonus. For each ablation we report validation and test BPC, throughput, peak memory, and the change relative to its matched full-model run. This design distinguishes the recurrent modular topology from routing regularizers.

Store-Distract-Query

We use a controlled online task to evaluate retention under interference. With five keys and ten values, the input vocabulary contains 50 key-value store tokens, ten distractor tokens, and five key-specific query tokens. Store events overwrite the selected key. Distractors update a running sum modulo ten. At a query, the target combines the most recently stored value, the distractor sum, and, in the Hard variant, per-key store and query counts. Loss and accuracy are computed only on query events.

Episode lengths are geometrically distributed with a curriculum-controlled mean. We report query accuracy by store-query gap, including failures on keys that have been overwritten, rather than aggregate token accuracy. All architectures use the same online sequence distribution, curriculum decisions, number of training tokens, and parameter budget.

Routing and Efficiency Analysis

For every configuration we report parameter count, recurrent-state scalars, training throughput, inference throughput at batch size one, and peak accelerator memory. Learned routes are summarized by per-layer entropy and average routing matrices. To test whether a route or column is functionally important rather than merely correlated with the output, we mask it at evaluation and measure the increase in BPC or query error. Interventions are computed on held-out data and aggregated across seeds.

Discussion

MoSAIC changes the allocation of recurrent capacity: the same number of trainable weights is distributed across several narrower persistent states connected by a learned routing graph. Parameter-matched improvement would therefore establish that this allocation is useful, but would not imply that it is free. In particular, increasing column count can enlarge the persistent activation state and introduces quadratic routing cost in C . State-, throughput-, and memory-aware comparisons are necessary to characterize this trade-off.

The architecture also does not guarantee functional specialization. Distinct parameters and participant identities make differentiated behavior possible, but attention visualizations alone cannot establish it. We use held-out route and column interventions to support only those specialization claims that produce repeatable causal effects.

Limitations.. MoSAIC remains sequential over time and therefore does not inherit the sequence-parallel training algorithms of modern linear recurrent models. The present natural-language evaluation is character-level and small-scale, and SDQ is synthetic. The grid exposes more recurrent state coordinates than some parameter-matched GRUs, so gains must be interpreted together with activation memory and throughput. Finally, routing regularizers introduce additional hyperparameters; their robustness across tasks remains an empirical question.

Conclusion

We introduced MoSAIC, a recurrent architecture that distributes a fixed parameter budget across independently parameterized GRU columns and learns their inputs through attention over bottom-up and delayed top-down message banks. This design separates persistent column states from an explicit recurrent communication graph while retaining constant state size with respect to sequence length. Our evaluation is designed to measure both predictive quality

and the state, throughput, and memory costs of this modular allocation.

References

- Arora, S., Eyuboglu, S., Timalsina, A., Johnson, I., Poli, M., Zou, J., Rudra, A., & Ré, C. (2023,). *Zoology: Measuring and Improving Recall in Efficient Language Models*.
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2024). xLSTM: Extended Long Short-Term Memory. *Arxiv Preprint Arxiv:2405.04517*.
- Cho, K., Merrienboer, B. van, Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724–1734.
- Dao, T., & Gu, A. (2024). Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. *Arxiv Preprint Arxiv:2405.21060*.
- Goyal, A., Didolkar, A., Lamb, A., Badola, K., Ke, N. R., Rahman, N., Binas, J., Blundell, C., Mozer, M., & Bengio, Y. (2022,). Coordination Among Neural Modules Through a Shared Global Workspace. *10th International Conference on Learning Representations (ICLR)*.
- Goyal, A., Lamb, A., Hoffmann, J., Sodhani, S., Levine, S., Bengio, Y., & Schölkopf, B. (2021,). Recurrent Independent Mechanisms. *9th International Conference on Learning Representations (ICLR)*.
- Graves, A., Fernandez, S., & Schmidhuber, J. (2007). Multi-Dimensional Recurrent Neural Networks. *Proceedings of the 17th International Conference on Artificial Neural Networks (ICANN)*, 549–558.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- Kalchbrenner, N., Danihelka, I., & Graves, A. (2016,). Grid Long Short-Term Memory. *3rd International Conference on Learning Representations (ICLR)*.
- Mittal, S., Lamb, A., Goyal, A., Voleti, V., Shanahan, M., Lajoie, G., Mozer, M., & Bengio, Y. (2020). Learning to Combine Top-Down and Bottom-Up Signals in Recurrent Neural Networks with Attention over Modules. *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 6972–6986.
- Morad, S. D., Kortvelesy, R., Bettini, M., Liwicki, S., & Prorok, A. (2023,). POPGym: Benchmarking Partially Observable Reinforcement Learning. *11th International Conference on Learning Representations (ICLR)*.
- Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., & De, S. (2023). Resurrecting Recurrent Neural Networks for Long Sequences. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 26670–26698.
- Qin, Z., Yang, S., Sun, W., Shen, X., Li, D., Li, W., & Zheng, Y. (2024,). HGRN2: Gated Linear RNNs with State Expansion. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Santoro, A., Faulkner, R., Raposo, D., Rae, J., Chrzanowski, M., Weber, T., Wierstra, D., Vinyals, O., Pascanu, R., & Lillicrap, T. (2018). Relational Recurrent Neural Networks. *Advances in Neural Information Processing Systems 31 (Neurips)*, 7310–7321.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (Neurips)*, 30.
- Yang, S., Wang, B., Zhang, Y., Shen, Y., & Kim, Y. (2024). Parallelizing Linear Transformers with the Delta Rule over Sequence Length. *Arxiv Preprint Arxiv:2406.06484*.